THE UNIVERSITY OF SYDNEY

MTRX3760 - Lab 3 Practice
Containers

These practice questions will not be graded. The graded portion of this lab will be released separately. The following will help you understand git, containers, and important aspects of system design and coding in C++. Please complete all practice questions.

P0. Git

If you have never used git before it might be helpful to follow the "Lab3 - Git Tutorial" available on Canvas. There are no marks for completing this tutorial, but there are marks in the graded portion of the lab (see separate handout) specifically linked to correct usage of git.

P1. Refactoring Hello Classes

In Lab 1, P4 had you split your Hello, Classes program across multiple files. In this question you will refactor that program to make it use constructors. Add your source files, as you completed them for Lab 1, to a git repo for Lab 3. Make sure the files are committed before proceeding with this question, as this will allow you to track your changes using `git diff`.

Refactor the code to use constructors and member initialiser lists. Make any additional improvements to the code that you'd like, e.g. improvements to structure or style based on what we've learned in lectures.

Add a line to the header comment of each modified file noting the date and nature of each change, e.g.
`// 2020-Sep-20 - Jane - Refactored to use constructors; improved variable naming`

Hint: Think about how to implement and commit these changes. It's better to have fine-grained git commits corresponding to atomic changes, rather than a single commit that changes many things at once.

Check your performance:
Look at your git commit history with the command

```
git log --oneline
```

Can you understand how each commit advanced the project in an atomic way from your git log?

THE UNIVERSITY OF SYDNEY

P2. Conveyor++

In Lab 1 you built up a set of classes to simulate the operation of a conveyor robot. In this question you will extend the functionality of this simulator to track individual items as they progress through the system. Begin with the unmodified example conveyor robot code provided with Lab 1, and as with P1 begin by committing the unchanged code to your repo so that you can inspect your changes.

The goals of this question are to become familiar with standard library containers and to explore the dynamics of a typical processing chain. Following the development process below, you will augment the conveyor to handle items with different processing times, and report associated key statistics.

Rather than counting objects, change your program to track individual objects as they progress through the system. Instantiate one object in memory for each item on the conveyor, and keep track of Items using a standard library container. The program should report the IDs of items as they are added to and removed from the conveyor. *Hint: You may find it easier to start by assigning each object a unique integer ID and tracking that, before moving on to instantiating an object for each item. You will need to create a new class Item, and use dynamic memory allocation to create and delete Item objects. The conveyor should keep pointers to instances, not the instances themselves.*

Assign each item a processing time, the amount of time it takes the processing robot to process it, measured in seconds as a floating point number. Assign each item a random processing time between 1.0 and 5.1 seconds, with a uniform probability distribution. *Hint: rational processing times are valid, not only integer processing times. Be careful how you choose your random numbers.*

There is a fixed time window C in which the robot processes as many items as possible. Add a function `ProcessItemsTimed` to the processing robot that processes as many items as possible within a fixed time window of C=16.1 seconds. Process items in their order of arrival - out-of-order processing appears in a later question. *Hints: an item cannot be partially processed during the fixed time window, so if 15.3 seconds have elapsed and the next item takes 2.0 seconds to process, the robot needs to sit idle until the end of the 16.1-second window, i.e. wait until the next ProcessItemsTimed call to process the item.*

Have your program calculate and report the following statistics:

- Average arm utilisation H: what percentage of time is the robot active, on average?
- Wait time E: on average, how long does an item sit on the belt waiting to be processed?

Also determine and include the following in your report:

- Instead of an arm capacity of C=16.1 seconds, determine C_{min}, the lowest value for which the number of items on the conveyor doesn't grow towards infinity.
- Maximum arm utilisation: what's the highest utilisation H you can achieve by adjusting the arm capacity C?



Find the following with the default arm capacity, C = 16.1 sec:

- The average number of items added in a cycle, *N*.
- The average processing time of an item, T_{ITEM}.
- The average active processing time per cycle $T = N\ T_{\text{ITEM}}$.
- Does the result *T* match up with the experimentally determined capacity C_{min}? Why?

P3. Prioritising Conveyor

In the conveyor example, we've been using a simple first-come, first-served approach to deciding the order in which items are processed. In this question you will explore two alternative approaches that prioritise either shorter or longer tasks. Copy your code from P2, and add functionality as required by this question.

Shortest job first: Implement a modified version of the conveyor robot from P2 that always takes the shortest job first, i.e. the item on the conveyor with the shortest processing time.

Longest job first: Similar to the above, have the arm always take the part with the longest processing time, not exceeding the available time, first.

Hint: The ability to switch between two operating modes is part of the functionality of this program, think about what code structure to use to support this functionality.

Check your understanding:

- For each approach, what are the average arm utilisation *H* and average wait time *E*?
- What is the maximum attainable arm utilisation for each approach?

To answer these questions, consider exploring the system's behaviour by devising and running experiments to get an intuition for what's happening.

ROS Tutorials

Complete the ROS tutorials here: http://wiki.ros.org/ROS/Tutorials. With your current knowledge you can complete up to and including Beginner Level Tutorial 10. By the end of Lab 4 you must have completed up to and including Beginner Level Tutorial 20.